12-Month DevOps Roadmap

Hands-On Edition v2 | 29 projects | CKA + AWS

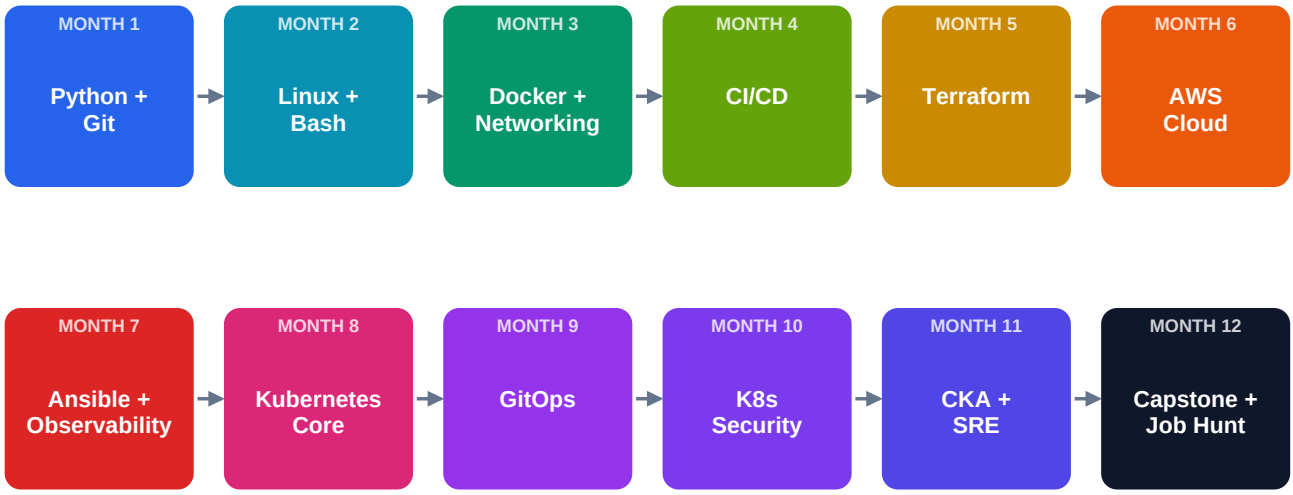
Python and Git > Linux > Docker > CI/CD > Terraform > AWS
Ansible > Observability > Kubernetes > GitOps > Security > Capstone

Prepared October 2026. Check tool versions and exam details at the linked official sources.

How to use this roadmap

This roadmap is DevOps only. It removes ML, LLM, GPU, Go, and multi-cloud so you can go deep on one set of skills that employers hire for. You already completed Phase 0 and most of Python basics, so Month 1 starts from where you stopped.

Pace: about 1.5 to 2 hours a day, 6 days a week, one rest day. Each month has 4 weeks, a project list, curated resources, and a done-when checklist.



Foundations (M1-M3) -> Automation (M4-M7) -> Kubernetes & Security (M8-M11) -> Prove it (M12)

The rules that make this work

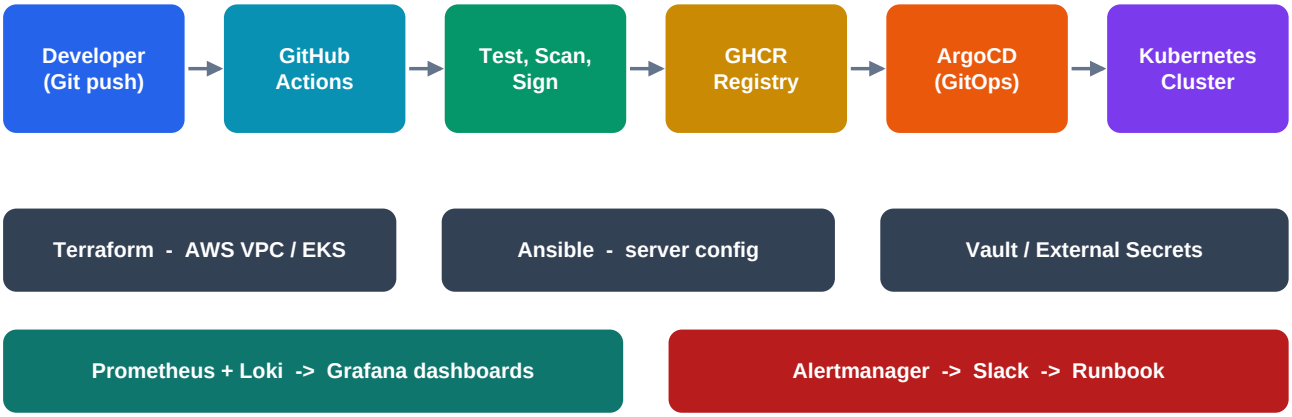
- **Project first, theory second.** Every week ends with something working in Git.
- **Document as you go.** README, diagram, and a 'what broke and how I fixed it' section for each project.
- **Break things on purpose** in your homelab. Troubleshooting is what CKA and interviews test.
- **Blog and portfolio from Month 1.** One short post per month plus your MkDocs portfolio site, updated as each project finishes.
- **Protect the daily habit.** 1.5-2 hours most days beats 5-hour weekend marathons.
- **One buffer week per quarter.** Use it strictly for catching up or deep rest, not for guilt.

Your starting point

- Done: Phase 0 (accounts, home lab, Ubuntu hardening, first blog post, hardening script).
- Done: Python core, functions, file I/O, error handling, modules, comprehensions.
- Remaining: OOP basics, venv, requests, env vars, CLI tools, pytest, structlog, subprocess, MkDocs site.

The DevOps pipeline you will build

By Month 12 you will have built every box in this diagram yourself. Each tool is introduced in the month where it fits naturally.



Month-by-month summary

Month	Focus	Projects	Main tools
1	Finish Python + Git Mastery	P1-P4	Python, Git, pytest
2	Linux & Bash Mastery	P5-P7	Linux, Bash, systemd
3	Networking + Docker	P8-P9	Docker, Compose, Trivy
4	CI/CD Pipelines	P10-P11	GitHub Actions, GHCR
5	Terraform (Infrastructure as Code)	P12-P13	Terraform, tflint, Infracost
6	Cloud Platform: AWS	P14-P15	AWS, IAM, VPC, Lambda
7	Ansible + Observability	P16-P17	Ansible, Prometheus, Grafana, Loki
8	Kubernetes Core	P18-P20	K3s, kubectl, Helm
9	GitOps with ArgoCD	P21-P23	ArgoCD, Argo Rollouts, Kustomize
10	Kubernetes Security & Policy	P24-P26	RBAC, Kyverno, Cosign, ESO
11	CKA Exam + SRE Practices	P27-P28	CKA, HPA, SLOs
12	Capstone + Job Hunt	P29	Everything combined

MONTH 1 OF 12

Finish Python + Git Mastery

Turn what you already know into real, shippable tools.

Week by week

Week 1	OOP basics (classes, __init__, methods), pip + venv, pyproject basics
Week 2	requests library + public APIs, JSON handling, env vars and .env files
Week 3	argparse / click CLIs, pytest first tests, structlog, subprocess module
Week 4	Git deep dive (branching, rebase, PRs), finish MkDocs homelab site, publish all projects

Hands-on projects

P1 System Info Reporter
CPU, memory, disk via psutil, pretty terminal output, --json flag. Add pytest tests.

P2 Log Parser CLI
Parse nginx/auth logs, count errors by hour, output a summary report. Use argparse or click.

P3 GitHub API Tool (CI foundation)
Call the GitHub API with a token from env vars. Add proper logging (structlog or logging), error handling for rate limits, and a pytest suite with mocked API responses. You will reuse this repo as the test project for CI in Month 4.

P4 Homelab Docs Site
MkDocs Material site with your ADRs, network diagram, and runbooks. Auto-deploy to GitHub Pages.

Special notes

- **Do not rush.** If OOP, venv, pytest, or click/argparse still feel shaky at the end of Week 3, take 3-5 extra days on them before the projects. Use your first buffer days for this.
- You already finished the basics, so do not re-watch beginner courses. Learn by building P1-P3 straight away.
- Use a branch + pull request for every project, even working alone. This builds the exact habit teams expect.
- Write a proper README for each project: what it does, how to install, example output.

Resources and links (click to open)

- Python official tutorial
- Automate the Boring Stuff (free)
- pytest docs
- Click docs (CLI)
- Pro Git book (free)
- Learn Git Branching (interactive)
- MkDocs Material
- freeCodeCamp (YouTube)

Done when

- [ ] 4 projects on GitHub with READMEs and passing tests
- [ ] P3 has logging and tests that pass without calling the real API
- [ ] Comfortable with git branch, merge, rebase, and PR workflow
- [ ] MkDocs site live on GitHub Pages

Blog + portfolio (monthly habit)

- **Post idea:** How I built a GitHub API tool with logging and tests
- Add this month's projects to your portfolio site (MkDocs on GitHub Pages) before moving on.

MONTH 2 OF 12

Linux & Bash Mastery

Servers are where DevOps lives. Get fast at the terminal.

Week by week

Week 1	Boot process, systemd, filesystems, LVM, users, permissions, ACLs
Week 2	Processes, signals, performance tools (top, iostat, vmstat), journalctl, logrotate
Week 3	Networking basics: TCP/IP, ss, tcpdump, curl, DNS, UFW/iptables, WireGuard
Week 4	Advanced Bash: functions, arrays, error handling, traps, cron and systemd timers

Hands-on projects

P5 Health Monitor with Alerts
Bash script checks CPU, RAM, disk, services. Sends alerts to Slack or Discord via webhook. Runs from a systemd timer.

P6 Backup Tool with Rotation
Daily/weekly/monthly backups with tar + rsync, checksums, retention policy, and a restore test.

P7 Custom systemd Service
Package one of your Python tools as a service with auto-restart, logging to journald, and resource limits.

Special notes

- Run ShellCheck on every script. It teaches good Bash faster than any book.
- Always use set -euo pipefail at the top of scripts, and understand why.
- Do OverTheWire Bandit levels as a daily 15-minute warm-up through the whole month.
- Take a VM snapshot before experiments. Breaking things is the goal, but recovering fast matters.

Resources and links (click to open)

- OverTheWire Bandit (Linux wargame)
- Linux Journey
- Bash Guide (Greg's Wiki)
- ShellCheck
- systemd.service man page
- roadmap.sh DevOps roadmap
- TechWorld with Nana (YouTube)

Done when

- [ ] Can debug a failing systemd service using only journalctl and systemctl
- [ ] 3 scripts passing ShellCheck with zero warnings
- [ ] Can explain what happens from power-on to login prompt

Blog + portfolio (monthly habit)

- **Post idea:** Building production-ready Bash scripts
- Add this month's projects to your portfolio site (MkDocs on GitHub Pages) before moving on.

MONTH 3 OF 12

Networking + Docker

Containers are the unit of delivery for everything that follows.

Week by week

Week 1	Networking for DevOps: subnets, CIDR, NAT, load balancers, TLS, reverse proxies
Week 2	Docker fundamentals: images, layers, volumes, networks, registries
Week 3	Multi-stage builds, image size optimisation, non-root users, healthchecks
Week 4	Docker Compose multi-service stacks, Trivy scanning, private registry

Hands-on projects

P8 Multi-stage Python Build

Take a FastAPI or Flask app from 1 GB to under 150 MB. Run as non-root. Document before/after sizes.

P9 Compose Full Stack

Nginx reverse proxy + app + PostgreSQL + Redis with healthchecks, named volumes, and an .env file. Add Trivy scan.

Special notes

- Never run containers as root and never bake secrets into images. Interviewers check for both.
- Learn layer caching: put rarely-changing steps first in your Dockerfile.
- Use docker history and dive to inspect what makes images large.

Resources and links (click to open)

- Docker docs
- Docker build best practices
- Docker Compose docs
- Trivy docs
- TechWorld with Nana (YouTube)
- KodeKloud CKA course + labs

Done when

- [ ] Image size cut by 60%+ with documented proof
- [ ] Compose stack survives docker compose down/up with data intact
- [ ] Trivy reports zero critical vulnerabilities on your final image

Blog + portfolio (monthly habit)

- **Post idea:** How I reduced my Docker image from 1 GB to 140 MB
- Add this month's projects to your portfolio site (MkDocs on GitHub Pages) before moving on.

MONTH 4 OF 12

CI/CD Pipelines

Every commit should be tested, built, scanned, and shipped automatically.

Week by week

Week 1	GitHub Actions basics: workflows, triggers, jobs, secrets, caching
Week 2	Automated tests + linting + matrix builds across Python versions
Week 3	Docker build, Trivy scan, push to GHCR, image tagging strategy
Week 4	Release automation, semantic versioning, environments with manual approval

Hands-on projects

P10 Full CI Pipeline

On every PR: lint, test, build image, scan with Trivy, push to GHCR on merge to main.

P11 Release Automation

Auto-generate changelog, version tags, and GitHub Releases using conventional commits and a release workflow.

Special notes

- Pin action versions to a commit SHA or full version, not a floating tag, for supply-chain safety.
- Use OIDC instead of long-lived cloud keys in pipelines once you reach AWS in Month 6.
- Keep pipelines under 10 minutes using caching and parallel jobs.

Resources and links (click to open)

- GitHub Actions docs
- Trivy docs
- Semantic Versioning spec
- TechWorld with Nana (YouTube)
- roadmap.sh DevOps roadmap

Done when

- [ ] Green pipeline badge on your repo README
- [ ] Failed tests or critical CVEs block the merge
- [ ] A tagged release is published without any manual steps

Blog + portfolio (monthly habit)

- **Post idea:** My CI pipeline from git push to scanned image
- Add this month's projects to your portfolio site (MkDocs on GitHub Pages) before moving on.

MONTH 5 OF 12

Terraform (Infrastructure as Code)

If it is not in code, it does not exist.

Week by week

Week 1	Terraform core: providers, resources, variables, outputs, state
Week 2	Modules, locals, for_each, data sources, lifecycle rules
Week 3	Remote state with locking, workspaces vs folders per environment
Week 4	Quality gates: validate, tflint, Infracost in pull requests

Hands-on projects

P12 Multi-Environment Terraform

dev and prod from shared modules, remote S3 state with DynamoDB (or native) locking, separate variable files.

P13 IaC Quality Pipeline

GitHub Actions runs fmt, validate, tflint, plan, and posts an Infracost cost comment on every PR.

Special notes

- **Cost rule:** set an AWS Budget alert at $5-10 BEFORE creating anything. Run terraform destroy at the end of every session, no exceptions.
- Set an AWS Budget alert at a few dollars BEFORE you create anything. Run terraform destroy after each session.
- Never commit .tfstate or secrets. Add them to .gitignore on day one.
- Never run apply without reading the plan. Make that a reflex.
- Keep modules small and single-purpose.

Resources and links (click to open)

- HashiCorp Terraform tutorials
- Terraform docs
- OpenTofu docs
- TFLint
- Infracost docs
- AWS Free Tier
- TechWorld with Nana (YouTube)

Done when

- [ ] Can destroy and rebuild a whole environment with one command
- [ ] Remote state with locking is working
- [ ] Every PR shows a cost estimate

Blog + portfolio (monthly habit)

- **Post idea:** Terraform state: what broke and how I fixed it
- Add this month's projects to your portfolio site (MkDocs on GitHub Pages) before moving on.

MONTH 6 OF 12

Cloud Platform: AWS

One cloud, learned deeply, transfers to all the others.

Week by week

Week 1	IAM, S3, VPC fundamentals: subnets, route tables, security groups
Week 2	EC2, ALB, Auto Scaling, RDS with Terraform
Week 3	Lambda, API Gateway, CloudWatch, EventBridge
Week 4	FinOps habits, Well-Architected review, SAA practice exams

Hands-on projects

P14 3-Tier Network on AWS

Custom VPC, public/private subnets, ALB, EC2 Auto Scaling group, RDS. 100% Terraform.

P15 Serverless App

Lambda + API Gateway + S3 + DynamoDB, deployed by CI using OIDC. Add CloudWatch alarms.

Special notes

- **Cost rule:** keep the $5-10 AWS Budget alert active. Prefer free-tier eligible and spot instances, and terraform destroy after every session.
- Prefer Azure? Swap in AZ-104 and follow the same project patterns on Azure. Do not mix clouds this year.
- Tag every resource (project, env, owner). It is the foundation of FinOps.
- Least privilege IAM from day one. Never use the root account for work, and enable MFA.

Resources and links (click to open)

- AWS Free Tier
- AWS Solutions Architect Associate exam
- AWS Well-Architected Framework
- AWS Budgets
- Terraform docs
- KodeKloud CKA course + labs

Done when

- [ ] Architecture diagram for P14 with cost estimate
- [ ] No long-lived access keys anywhere in your repos
- [ ] Scoring 75%+ on SAA practice exams (if taking it)
- [ ] AWS Budget alert exists and terraform destroy is a habit

Blog + portfolio (monthly habit)

- **Post idea:** A 3-tier AWS network for under $5
- Add this month's projects to your portfolio site (MkDocs on GitHub Pages) before moving on.

Interview practice (30-45 min per week, out loud)

- Explain out loud what happens when you type a URL and press Enter (DNS, TCP, TLS, load balancer, server). Then: how do you fix a locked or drifted Terraform state?

MONTH 7 OF 12

Ansible + Observability

Configure servers consistently, then watch them like a hawk.

Week by week

Week 1	Ansible fundamentals: inventory, playbooks, modules, variables
Week 2	Roles, templates (Jinja2), handlers, Ansible Vault, idempotency
Week 3	Prometheus + node_exporter + PromQL basics
Week 4	Grafana dashboards, Loki logs, Alertmanager routing to Slack

Hands-on projects

P16 Ansible Server Baseline

Roles that harden SSH, install Docker, configure firewall and users on fresh VMs. Tested with Molecule.

P17 Full Observability Stack

Prometheus, Grafana, Loki, Alertmanager via Docker Compose or Helm. Dashboards for system, app, and logs plus 5 meaningful alerts.

Special notes

- Run every playbook twice. The second run must show zero changes (idempotency).
- Alert on symptoms users feel (errors, latency), not only on CPU.
- Every alert needs a runbook link explaining what to do.

Resources and links (click to open)

- Ansible docs
- Molecule (role testing)
- Prometheus docs
- Grafana docs
- Grafana Loki docs
- TechWorld with Nana (YouTube)

Done when

- [ ] Fresh VM goes from nothing to production-ready with one command
- [ ] Deliberately kill a service and watch the alert fire in Slack

Blog + portfolio (monthly habit)

- **Post idea:** Alerts that matter: my observability stack
- Add this month's projects to your portfolio site (MkDocs on GitHub Pages) before moving on.

Interview practice (30-45 min per week, out loud)

- Walk me through a pipeline you built (P10 or P17). What would you do to make it faster and safer?

MONTH 8 OF 12

Kubernetes Core

The centre of modern DevOps. Take your time here.

Week by week

Week 1	Architecture, Pods, Deployments, Services, kubectl fluency
Week 2	ConfigMaps, Secrets, volumes, StatefulSets, Ingress
Week 3	Build your own K3s or kubeadm cluster on homelab VMs
Week 4	Helm charts: write one from scratch for your own app

Hands-on projects

P18 Home Kubernetes Cluster

Multi-node K3s or kubeadm cluster with an ingress controller, cert-manager, and persistent storage.

P19 Microservices on K8s

3-service app with Ingress routing, ConfigMaps, Secrets, and a PersistentVolume for the database.

P20 Your Own Helm Chart

Reusable chart with values for dev/prod, probes, resource limits, and helm test.

Special notes

- Use kubectl imperative commands and --dry-run=client -o yaml to generate manifests fast. This is critical for the CKA time limit.
- Learn to read kubectl describe and events first. 80% of problems are explained there.
- Always set resource requests and limits and add readiness/liveness probes.

Resources and links (click to open)

- Kubernetes docs
- Kubernetes The Hard Way
- K3s docs
- Helm docs
- KodeKloud CKA course + labs
- killer.sh exam simulator
- TechWorld with Nana (YouTube)

Done when

- [ ] Can deploy, expose, scale, and troubleshoot an app without copy-pasting from docs
- [ ] Helm chart installs cleanly on a fresh namespace
- [ ] Cluster survives a node reboot

Blog + portfolio (monthly habit)

- **Post idea:** Building a Kubernetes cluster at home
- Add this month's projects to your portfolio site (MkDocs on GitHub Pages) before moving on.

Interview practice (30-45 min per week, out loud)

- Debug a pod in CrashLoopBackOff step by step: describe, logs, previous logs, events, probes, resources.

MONTH 9 OF 12

GitOps with ArgoCD

Git becomes the single source of truth for your cluster.

Week by week

Week 1	GitOps principles, ArgoCD install, first Application
Week 2	Kustomize bases and overlays for dev/staging/prod
Week 3	Argo Rollouts: canary and blue-green with automated analysis
Week 4	ApplicationSets, App-of-Apps pattern, repo structure best practice

Hands-on projects

P21 ArgoCD Deployment
Deploy your Helm chart (P20) from Git with auto-sync, self-heal, and pruning.

P22 Canary Release
Argo Rollouts shifts 10%, 50%, 100% traffic with automatic rollback on error-rate spike from Prometheus.

P23 Environment Overlays
Kustomize overlays for dev and prod, deployed by separate ArgoCD apps.

Special notes

- Separate your app code repo from your config/manifests repo. This is standard practice.
- Never kubectl apply by hand into a GitOps cluster. Change Git, then watch ArgoCD sync.
- Practise a rollback by simply reverting a Git commit.

Resources and links (click to open)

- OpenGitOps principles
- ArgoCD docs
- Argo Rollouts docs
- Kustomize
- Helm docs
- TechWorld with Nana (YouTube)

Done when

- [ ] Merging a PR is the only way a change reaches the cluster
- [ ] A bad release auto-rolls back without human action

Blog + portfolio (monthly habit)

- **Post idea:** Why I stopped using kubectl apply (GitOps)
- Add this month's projects to your portfolio site (MkDocs on GitHub Pages) before moving on.

Capstone scaffolding

- Decide the exact capstone architecture now. Create the capstone repo, write the first ADRs (why EKS vs K3s, why ArgoCD, why Kyverno) and a high-level README with a draft diagram.

Interview practice (30-45 min per week, out loud)

- Explain GitOps and how you roll back a bad release. Why separate app repo and config repo?

MONTH 10 OF 12

Kubernetes Security & Policy

Security is now part of every DevOps job description.

Week by week

Week 1	RBAC, ServiceAccounts, least privilege, Pod Security Standards
Week 2	NetworkPolicies, default-deny, namespace isolation
Week 3	External Secrets Operator + Vault or AWS Secrets Manager
Week 4	Kyverno policies, Cosign image signing, SBOMs and verification

Hands-on projects

P24 Cluster Hardening
RBAC roles per team, default-deny NetworkPolicies, restricted Pod Security in every namespace.

P25 Secrets Management
No secrets in Git. External Secrets syncs from a vault into the cluster.

P26 Policy + Supply Chain
Kyverno blocks unsigned images, latest tags, and root containers. Cosign signs images in CI.

Special notes

- **Book the CKA exam at the start of this month** for a date in Month 11. A fixed date creates urgency and daily practice. Do killer.sh session #1 this month.
- Test policies in audit mode first, then switch to enforce. Otherwise you will lock yourself out.
- Prove each control works: show the blocked request in your README.
- Run kube-bench or Trivy cluster scans and fix what they report.
- **Start applying lightly this month.** Intermediate projects (CI, Docker, Terraform, Kubernetes) already get interviews. Aim for 3-5 applications a week while you finish the capstone.

Resources and links (click to open)

- K8s RBAC docs
- Pod Security Standards
- Network Policies
- External Secrets Operator
- Kyverno docs
- Sigstore Cosign
- Kubernetes docs

Done when

- [ ] Attempting a root or unsigned container is rejected
- [ ] You can explain every RBAC rule you wrote
- [ ] CKA exam booked and killer.sh session #1 completed
- [ ] Light job applications started

Blog + portfolio (monthly habit)

- **Post idea:** Locking down a Kubernetes cluster step by step
- Add this month's projects to your portfolio site (MkDocs on GitHub Pages) before moving on.

Capstone scaffolding

- Keep the capstone repo moving: finish the ADRs, add the architecture diagram to the README, and write the Terraform skeleton and folder structure. Month 12 should be assembling, not designing.

Interview practice (30-45 min per week, out loud)

- How do you manage secrets, RBAC, and policy in a cluster? What is default-deny and why use it?

MONTH 11 OF 12

CKA Exam + SRE Practices

Certify your Kubernetes skills and learn to operate under pressure.

Week by week

Week 1	CKA domain review: cluster setup, upgrades, etcd backup/restore
Week 2	Troubleshooting drills: nodes, networking, scheduling, storage
Week 3	killer.sh simulator (twice), time-boxed practice every day
Week 4	Take the CKA. Then SLOs, HPA, and the incident drill project.

Hands-on projects

P27 SLO Dashboard + HPA

Define SLIs/SLOs for your app, build the Grafana dashboard, and add HPA based on load tests.

P28 Incident Drill + Postmortem

Break your cluster on purpose, debug with only logs and metrics, then write a blameless postmortem.

Special notes

- Do killer.sh session #2 right before the exam (you did #1 in Month 10). Review every question you missed.
- The CKA is a hands-on terminal exam. Speed with kubectl, vim, and kubernetes.io search is more important than memorisation.
- You may open the official Kubernetes docs during the exam. Practise navigating them quickly.
- Check the current CKA curriculum and exam rules on the Linux Foundation page before you start, since versions and details change.

Resources and links (click to open)

- CKA exam page (Linux Foundation)
- killer.sh exam simulator
- KodeKloud CKA course + labs
- Google SRE Book (free)
- DORA metrics
- Kubernetes docs

Done when

- [ ] CKA passed
- [ ] Postmortem published on your blog
- [ ] Etcd backup and restore done from memory

Blog + portfolio (monthly habit)

- **Post idea:** My CKA preparation and my first postmortem
- Add this month's projects to your portfolio site (MkDocs on GitHub Pages) before moving on.

Interview practice (30-45 min per week, out loud)

- Record yourself answering 5 questions in a row, then re-listen. Add one real incident story from your P28 drill.

MONTH 12 OF 12

Capstone + Job Hunt

Combine everything into one project that gets you hired.

Week by week

Week 1	Capstone: Terraform infra + CI pipeline + registry
Week 2	Capstone: ArgoCD, canary, secrets, policies
Week 3	Capstone: observability, cost dashboards, docs, demo video
Week 4	Resume, portfolio, mock interviews, applications

Hands-on projects

P29 CAPSTONE: Production-Style Platform

Full pipeline from Git push to running, monitored, secured workload. See the architecture on the next page.

Special notes

- Start applying in Month 10. Do not wait for perfection.
- Quantify results on your resume: build time cut 70%, image size down 85%, deploy time from 20 min to 3 min.
- Prepare to explain every design decision in the capstone. Interviewers will ask why.
- Practise answering: how would you troubleshoot a pod stuck in CrashLoopBackOff?

Resources and links (click to open)

- roadmap.sh DevOps roadmap
- Architecture Decision Records
- Excalidraw (diagrams)
- Diagrams as Code (Python)
- Dev.to
- Hashnode
- Google SRE Book (free)

Done when

- [ ] Capstone repo with README, diagram, ADRs, runbook, and demo video
- [ ] Resume + LinkedIn + portfolio live
- [ ] 5-10 targeted applications per day

Blog + portfolio (monthly habit)

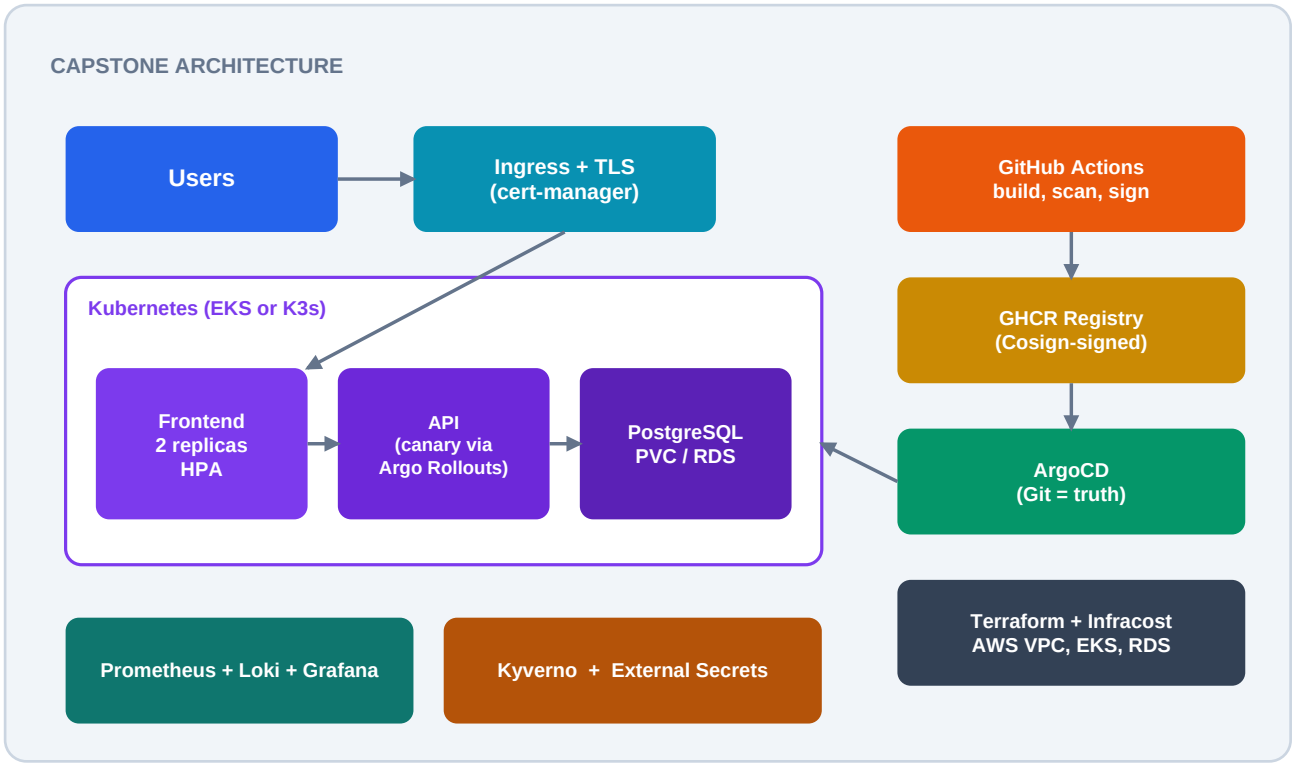
- **Post idea:** Capstone write-up: from git push to production
- Add this month's projects to your portfolio site (MkDocs on GitHub Pages) before moving on.

Interview practice (30-45 min per week, out loud)

- Do 2-3 full mock interviews with a friend, a community, or a recording. Practise the capstone walkthrough in 5 minutes.

Capstone architecture

Build this end to end. Every component below comes from a previous month.



Capstone README must include

- Architecture diagram (like the one above, made with Excalidraw or diagrams-as-code)
- Architecture Decision Records explaining why you chose each tool
- Runbook for the top 3 failure scenarios
- Cost breakdown from Infracost
- 5-minute demo video: push code, watch it deploy, trigger a canary, show an alert

Execution advice

What actually makes people succeed

- **Protect the daily habit** more than the perfect plan. 1.5-2 hours most days beats 5-hour weekend marathons.
- **Use the quarterly buffer week** strictly for catching up or deep rest, not for guilt.
- **Add a 'what broke and how I fixed it' section** to every project README. This is interview gold.
- **Treat every project as if a senior engineer will review it:** clean commits, good branch names, proper pull requests, even when working alone.
- **Homelab safety:** keep at least one stable VM snapshot or cloud backup so you can break things freely.
- **Track progress visually** with the tracker on the next page, or a Notion/Trello board. Seeing months fill up is motivating.

Job-readiness checkpoint: end of Month 9-10

- [ ] Several public GitHub repos with real CI, Docker, Terraform, and Kubernetes
- [ ] A live MkDocs or portfolio site with your projects
- [ ] At least 4-5 technical blog posts published
- [ ] Evidence you can troubleshoot: postmortems and runbooks
- [ ] Capstone architecture decided, ADRs and high-level README started
- [ ] Weekly out-loud interview practice running since Month 6

Start applying lightly from Month 10, even if the capstone is not finished. Many people get interviews from strong intermediate projects.

Progress tracker

Tick each box as you finish it. Print this page or copy it into Notion, Trello, or a Markdown file.

Month	Focus	Projects	Blog post	Portfolio updated	Done
1	Finish Python + Git Mastery	P1-P4	[ ]	[ ]	[ ]
2	Linux & Bash Mastery	P5-P7	[ ]	[ ]	[ ]
3	Networking + Docker	P8-P9	[ ]	[ ]	[ ]
4	CI/CD Pipelines	P10-P11	[ ]	[ ]	[ ]
5	Terraform (Infrastructure as Code)	P12-P13	[ ]	[ ]	[ ]
6	Cloud Platform: AWS	P14-P15	[ ]	[ ]	[ ]
7	Ansible + Observability	P16-P17	[ ]	[ ]	[ ]
8	Kubernetes Core	P18-P20	[ ]	[ ]	[ ]
9	GitOps with ArgoCD	P21-P23	[ ]	[ ]	[ ]
10	Kubernetes Security & Policy	P24-P26	[ ]	[ ]	[ ]
11	CKA Exam + SRE Practices	P27-P28	[ ]	[ ]	[ ]
12	Capstone + Job Hunt	P29	[ ]	[ ]	[ ]

Key dates to set: AWS Budget alert (Month 5), interview practice starts (Month 6), book CKA (start of Month 10), killer.sh #1 (Month 10), start applying (Month 10), killer.sh #2 and CKA exam (Month 11).

Certifications

Certification	When	Priority	Link
CKA - Certified Kubernetes Administrator	Book: start of Month 10. Sit: Month 11	Must-have	Linux Foundation
AWS Solutions Architect Associate	Month 6 (study alongside)	Recommended	AWS
CKAD / CKS / AZ-400	After landing a job	Optional	-

Exam fees, formats, and retake rules change. Always confirm on the official pages above.

Interview prep checklist

- Explain what happens when you type a URL and press Enter (DNS, TCP, TLS, load balancer, pod).
- Walk through a CI/CD pipeline you built and how you would make it faster and safer.
- Troubleshoot: pod in CrashLoopBackOff, node NotReady, 502 from Ingress, Terraform state drift.
- Explain the difference between a Deployment, StatefulSet, and DaemonSet.
- Describe a production incident (your P28 drill) and what you changed afterwards.
- Explain how you manage secrets, cost, and least privilege.

Best free practice and community links

- roadmap.sh DevOps roadmap
- Kubernetes The Hard Way
- OverTheWire Bandit (Linux wargame)
- Google SRE Book (free)
- TechWorld with Nana (YouTube)
- freeCodeCamp (YouTube)
- KodeKloud CKA course + labs
- killer.sh exam simulator
- Dev.to
- Hashnode

After the job: what comes next

- Go for Kubernetes operators and CLI tools
- CKS (security) and a second cloud such as Azure
- The MLOps layer: ML basics, GPU orchestration, vLLM, MLflow. Your DevOps base carries straight into it.

Link note: all links point to official project sites or long-standing community resources, but websites move. If a link breaks, search the tool name plus 'official docs'.